The CUDA kernels of RutticeQCD, one by one

Wei-Lun Chen

October 24, 2026 (internal note)

For every kernel: what it computes, written out line by line, how its threads are laid out, and what in memory decides its speed. File names are under `cuda_bridge/include/`. The communication kernels are in the note on several GPUs.

Contents

1 How the fields lie in memory **2**
1.1 Blocks of 32 sites . . . 2
1.2 Spinors with a fifth dimension . . . 2
1.3 Links: two rows are enough . . . 3

2 The domain-wall operator **3**
2.1 What the operator is . . . 3
2.2 `fifth_term`: $a\psi_s + y(Y\psi)_s$ . . . 3
2.3 `hop_kernel`: out = [from−] $H(b - cY)$ in . . . 3
2.4 `mdwf_dslash_1pass_kernel`: all of D in one pass . . . 4
2.5 `mdwf_fifth_kernel`: $(a_1 + y_1 Y)x + (a_2 + y_2 Y)z$ . . . 4
2.6 The fifth dimension as a matrix: `fifth_gemm` . . . 4
2.7 `hop_gemm_kernel`: hops, then a matrix, in one pass . . . 5
2.8 Small kernels . . . 5

3 Wilson and clover **5**
3.1 `wilson_interior_kernel` . . . 5
3.2 `clover_term_kernel`: $T\psi$ . . . 5
3.3 `clover_dslash_kernel` . . . 6

4 The fermion force: `hop_force_kernel` **6**

5 The gauge field **6**
5.1 Layout . . . 6
5.2 `stout_step_kernel`: $V = e^{iQ}U$ . . . 6
5.3 `stout_chain_local_kernel` and `stout_chain_staples_kernel` . . . 7
5.4 `gauge_force_kernel` and `gauge_traces_kernel` . . . 7
5.5 `links_exp_update_kernel`: $U \leftarrow e^{\epsilon H}U$ . . . 7
5.6 Other small kernels . . . 7

6 The multigrid: P **7**
6.1 What P holds . . . 7
6.2 `fold_restrict_kernel`: coarse $= P^\dagger$fine . . . 8
6.3 `fold_prolong_kernel`: fine $= P$coarse . . . 8
6.4 Orthonormalising P . . . 8

7 The multigrid: the coarse operator **9**
7.1 Probing: `hop_split_kernel`, `coarse_units`, `coarse_columns` . . . 9
7.2 `coarse_pass_kernel` . . . 9
7.3 `blocks_inverse_kernel`: Gauss–Jordan . . . 9
7.4 `blocks_abs_inverse_gamma5_kernel`: $|H|^{-1}$ by Jacobi . . . 9
7.5 The exact adjoint: `coarse_adjoint_kernel` . . . 10

8 The levels below the coarse lattice **10**

9 Linear algebra: `blas.cuh` **10**
9.1 Fused operations . . . 10
9.2 Deterministic reductions . . . 10
9.3 Sets of vectors . . . 10

1 How the fields lie in memory

1.1 Blocks of 32 sites

A field has N_c reals per site. Sites are grouped by 32, and inside a group the same component of the 32 sites is consecutive:

$$a(c,n) = \big(\lfloor n/32 \rfloor \cdot N_c + c\big) \cdot 32 + (n \bmod 32). \tag{1.1}$$

Why: a warp is 32 threads. If thread j handles site $32k+j$ and reads component c,

$$a(c, 32k+j) = (kN_c + c) \cdot 32 + j, \qquad j = 0, \ldots, 31, \tag{1.2}$$

32 consecutive numbers: one memory transaction. This is the only rule behind every layout below.

1.2 Spinors with a fifth dimension

A domain-wall spinor has L_s spinors of 24 reals per site. Component (colour k, spin a, real/imag r) at fifth coordinate s is component

$$c = 24s + 2(k + 3a) + r \tag{1.3}$$

of the site, $N_c = 24L_s$. A field on one parity numbers its sites by $\lfloor x/2 \rfloor$:

$$n_{\text{half}} = \lfloor x/2 \rfloor + \tfrac{L_x}{2}\big(y + L_y(z + L_z t)\big), \qquad x = 2\lfloor x/2 \rfloor + \big((p+y+z+t) \bmod 2\big), \tag{1.4}$$

the second line recovering x from the parity p. Several vectors of a field lie one after another, each padded to a whole number of blocks.

1.3 Links: two rows are enough

A link $U \in SU(3)$ has rows r_0, r_1, r_2. Unitarity and $\det U = 1$ give

$$r_2 = \overline{r_0 \times r_1}, \tag{1.5}$$

$$(r_2)_c = \overline{(r_0)_a (r_1)_b - (r_0)_b (r_1)_a}, \qquad a = c+1,\ b = c+2 \pmod 3. \tag{1.6}$$

So the operators keep 12 reals per link instead of 18 when the links are in $SU(3)$, and rebuild row 2 in registers (`load_link` in `hops.cuh`). A hop reads mostly links, so this saves about a third of what it reads.

2 The domain-wall operator

2.1 What the operator is

The Möbius operator on ψ_s, $s = 0, \ldots, L_s - 1$, is

$$D\psi_s = (b\, D_W + 1)\psi_s + (c\, D_W - 1)(P_- \psi_{s+1} + P_+ \psi_{s-1}), \tag{2.1}$$

with $D_W = (4 - M_5) - \frac{1}{2}\tilde{H}$ and $\tilde{H}$ the hops without κ. Define the fifth-dimension hop

$$(Y\psi)_s = -P_- \psi_{s+1} - P_+ \psi_{s-1} \qquad (+m \text{ in place of } -1 \text{ across the walls}). \tag{2.2}$$

Then $P_- \psi_{s+1} + P_+ \psi_{s-1} = -(Y\psi)_s$, and

$$D\psi = (bD_W + 1)\psi - (cD_W - 1)Y\psi \tag{2.3}$$

$$= \big[b(4 - M_5) + 1\big]\psi + \big[1 - c(4 - M_5)\big]Y\psi - \tfrac{1}{2}\tilde{H}\,(b\psi - cY\psi) \tag{2.4}$$

$$= \underbrace{(d_+ + d_- Y)\psi}_{C\psi} - \tfrac{1}{2}\tilde{H}\,(b - cY)\psi, \tag{2.5}$$

with $d_+ = 1 + b(4 - M_5)$ and $d_- = 1 - c(4 - M_5)$. C acts within a four-dimensional site; the hops act on $w = (b - cY)\psi$. Every kernel below is a piece of this line.

2.2 `fifth_term`: $a\psi_s + y(Y\psi)_s$

$$w_s = a\,\psi_s + y\,(Y\psi)_s \tag{2.6}$$

$$= a\,\psi_s - y\,P_- \psi_{s+1} - y\,P_+ \psi_{s-1} \qquad (0 < s < L_s - 1), \tag{2.7}$$

$$w_0 = a\,\psi_0 - y\,P_- \psi_1 + y\,m\,P_+ \psi_{L_s - 1}, \tag{2.8}$$

$$w_{L_s-1} = a\,\psi_{L_s-1} + y\,m\,P_- \psi_0 - y\,P_+ \psi_{L_s-2}. \tag{2.9}$$

For $Y^\dagger$ the projectors swap: P_+ from above, P_- from below.

2.3 `hop_kernel`: out $= [\mathbf{from}-]\, H(b - cY)$ in

At output site x and fifth coordinate s:

$$\mathrm{out}_s(x) = -\kappa \sum_\mu \Big[(1 - s_\pm \gamma_\mu) U_\mu(x)\, w_s(x + \hat{\mu}) + (1 + s_\pm \gamma_\mu) U_\mu(x - \hat{\mu})^\dagger\, w_s(x - \hat{\mu})\Big], \tag{2.10}$$

with w_s from `fifth_term` computed on the fly at the neighbour ($a = b$, $y = -c$) and $s_\pm = \pm 1$ for H, $H^\dagger$. A leg is done in three steps (`leg_half`, link, `add_leg`). For $\mu = 0$ (Sec. 1.1 of the other note):

$$h_0 = w_0 + i s_\pm w_3, \qquad h_1 = w_1 + i s_\pm w_2, \tag{2.11}$$

$$t_a = U\, h_a \quad (a = 0, 1), \tag{2.12}$$

$$\mathrm{out}_0 \mathrel{+}= -\kappa\, t_0, \qquad \mathrm{out}_1 \mathrel{+}= -\kappa\, t_1, \tag{2.13}$$

$$\mathrm{out}_3 \mathrel{+}= -\kappa\,(-i s_\pm)\, t_0, \qquad \mathrm{out}_2 \mathrel{+}= -\kappa\,(-i s_\pm)\, t_1. \tag{2.14}$$

Two $SU(3)$ products per leg instead of four.

Threads. A thread for each (site, fibre), a fibre being one s of one vector. The threads of a block are 2D: `threadIdx.x` the site (consecutive sites: coalesced by Eq. (1.1)), `threadIdx.y` the fibre. All fibres of a site read the same eight links, which then come from the cache after the first. At most 256 threads a block: a thread may use 255 registers and a block has 65536.

Double precision. The two legs of a direction are done one after the other, so only one spinor and one link are live: 168 registers, three blocks per multiprocessor. The sums go into `out` in the same order either way.

2.4 `mdwf_dslash_1pass_kernel`: all of D in one pass

$$\mathrm{out}_s(x) = \underbrace{d_+\psi_s(x) + d_-(Y\psi)_s(x)}_{\texttt{fifth_term}(a=d_+,\,y=d_-)} + \mathrm{hops}(\kappa=\tfrac{1}{2}). \tag{2.15}$$

One read of ψ and the links, one write of out. Used for D on the whole lattice.

2.5 `mdwf_fifth_kernel`: $(a_1 + y_1 Y)x + (a_2 + y_2 Y)z$

Two `fifth_term`s added, for the parts of the even-odd operators that act within a site.

2.6 The fifth dimension as a matrix: `fifth_gemm`

In the Dirac representation γ_5 swaps the upper and lower spin pairs:

$$\gamma_5 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad P_\pm = \tfrac{1}{2}(1 \pm \gamma_5) = \tfrac{1}{2}\begin{pmatrix} 1 & \pm 1 \\ \pm 1 & 1 \end{pmatrix}. \tag{2.16}$$

Write a spinor at s as (upper u_s, lower l_s). Any operator of the form $\sum_{s'}(A_{ss'}P_+ + B_{ss'}P_-)$ gives

$$\mathrm{out}^u_s = \sum_{s'} \left[A_{ss'} \tfrac{u_{s'}+l_{s'}}{2} + B_{ss'} \tfrac{u_{s'}-l_{s'}}{2} \right] = \sum_{s'} \left[\tfrac{A+B}{2} u_{s'} + \tfrac{A-B}{2} l_{s'} \right]_{ss'}, \tag{2.17}$$

$$\mathrm{out}^l_s = \sum_{s'} \left[A_{ss'} \tfrac{u_{s'}+l_{s'}}{2} - B_{ss'} \tfrac{u_{s'}-l_{s'}}{2} \right] = \sum_{s'} \left[\tfrac{A-B}{2} u_{s'} + \tfrac{A+B}{2} l_{s'} \right]_{ss'}. \tag{2.18}$$

So the operator is one real $2L_s \times 2L_s$ matrix M acting on the rows $(u_0, l_0, u_1, l_1, \dots)$. Y, C, C^{-1}, $b - cY$, Neff's Q, their adjoints and their products are all of this form. Products are formed on the host, and one kernel applies the result. In a block of 32 sites the rows of the field are $2L_s$ rows of $12 \times 32 = 384$ reals:

$$\mathrm{out}\,(2L_s \times 384) = M\,(2L_s \times 2L_s)\,\mathrm{in}\,(2L_s \times 384). \tag{2.19}$$

Double precision on the tensor cores (`fifth_gemm_mma_kernel`). The instruction `mma.m8n8k4.f64` computes $D(8\times 8)$ += $A(8\times 4)\,B(4\times 8)$; thread t of a warp holds $A[t/4][t \bmod 4]$, $B[t \bmod 4][t/4]$ and two entries of D. A warp takes a strip of 8 columns; for every 4 rows of the input it reads its 4×8 reals once and multiplies them with the tiles of M.

Why the XOR. M sits in shared memory, which has 32 banks of 4 bytes. A double occupies two banks, so double number k of a row of length K_{16} (a multiple of 16) is in banks

$$\mathrm{bank}(row, k) = 2(row \cdot K_{16} + k) \bmod 32 = 2k \bmod 32 \qquad (2K_{16} \equiv 0 \bmod 32). \tag{2.20}$$

A tile read takes rows 0..7 and columns $k_0..k_0+3$: all eight rows hit the same banks, an eight-way conflict. Store (row, k) at column $k' = k \oplus 4(row \bmod 4)$:

$$row \bmod 4 = 0 : k' \in \{k_0..k_0+3\}, \qquad row \bmod 4 = 1 : k' \in \{k_0..k_0+3\} \oplus 4, \tag{2.21}$$

$$row \bmod 4 = 2 : k' \in \{k_0..k_0+3\} \oplus 8, \qquad row \bmod 4 = 3 : k' \in \{k_0..k_0+3\} \oplus 12, \tag{2.22}$$

four disjoint groups of four doubles, $4 \times 4 \times 2 = 32$ banks. Rows r and $r+4$ still share, a two-way conflict. That is the least possible: 32 threads reading 8 bytes need 256 bytes, two passes of 128.

Single precision (`fifth_gemm_columns_kernel`): a thread per column of a block; it reads the $2L_s$ reals of its column once and keeps the $8N_T$ results in registers. All threads read the same entry of M at the same time, a broadcast. `fifth_gemm_tf32_kernel` does the product on the tensor cores in TF32 (10 bits of mantissa, relative error 10^{-3}), only for operators whose result is corrected afterwards (smoothers, inner solves).

2.7 `hop_gemm_kernel`: hops, then a matrix, in one pass

out = [from−] $M\,H(b - cY)$ in. The hops of a block of sites are written into shared memory as the $2L_s$ rows of the block, then multiplied by M as above. In double precision the `mma` reads 4 rows $j..j+3$ and 8 columns of the tile. A row is 12×(sites of the block) doubles; when that is a multiple of 16 doubles (128 bytes), rows j and $j+1$ hit the same banks. Storing column k of row j at $k \oplus 8(j \bmod 2)$ shifts the odd rows by 8 doubles = 16 banks:

$$\text{bank}(j,k) = 2\big(k \oplus 8(j \bmod 2)\big) \bmod 32, \tag{2.23}$$

so even and odd rows use the two halves of the banks; rows j and $j+2$ share, two-way, again the least for 32 doubles. This is the XOR seen in the code; the face buffers of several GPUs need none (they are in global memory).

2.8 Small kernels

$$\texttt{gamma5}: \quad \text{out}_s = \gamma_5\,\text{in}_s, \text{ or with reflection } \text{out}_s = \gamma_5\,\text{in}_{L_s-1-s}\ (\Gamma_5 = \gamma_5 R), \tag{2.24}$$

$$\texttt{walls}(\to 4d): \quad q = P_-\psi_{s_-} + P_+\psi_{s_+}, \qquad (s_-, s_+) = (0, L_s-1) \text{ for the propagator}, \tag{2.25}$$

$$\texttt{walls}(\leftarrow 4d): \quad \psi_{s_-} = P_-\eta,\ \psi_{s_+} = P_+\eta, \tag{2.26}$$

$$\texttt{parity_copy}: \quad \text{half}[n_{\text{half}}] \leftrightarrow \text{whole}[n]. \tag{2.27}$$

3 Wilson and clover

3.1 `wilson_interior_kernel`

$D\psi = \psi + H\psi$ with the hops of Sec. 2.3, $L_s = 1$, $b = 1$, $c = 0$. The fibres of a site are its vectors, which share the links.

3.2 `clover_term_kernel`: $T\psi$

$T = 1 - \kappa c_{SW}\sigma \cdot F$ commutes with γ_5. In the Dirac representation the eigenvectors of γ_5 are $(e_a \pm e_{a+2})/\sqrt{2}$, so in that basis T is two Hermitian 6×6 blocks (index colour +3×spin):

$$\psi^{\pm}_{k+3a} = \psi_{a,k} \pm \psi_{a+2,k}, \qquad a = 0,1 \quad (\text{the } 1/\sqrt{2} \text{ left out}), \tag{3.1}$$

$$\phi^{\pm} = T_{\pm}\,\psi^{\pm}, \tag{3.2}$$

$$(T\psi)_{a,k} = \tfrac{1}{2}(\phi^{+}_{k+3a} + \phi^{-}_{k+3a}), \qquad (T\psi)_{a+2,k} = \tfrac{1}{2}(\phi^{+}_{k+3a} - \phi^{-}_{k+3a}), \tag{3.3}$$

the two $1/\sqrt{2}$ making the $\frac{1}{2}$. A Hermitian 6×6 block is stored packed: 6 real diagonal entries and 15 complex above the diagonal,

$$6 + 2 \cdot 15 = 36 \text{ reals}, \qquad 2 \text{ blocks} = 72 \text{ reals a site}. \tag{3.4}$$

The same kernel applies T^{-1} (its blocks inverted on the host).

3.3 `clover_dslash_kernel`

$D\psi = T\psi + H\psi$ in one pass.

4 The fermion force: `hop_force_kernel`

For $\delta U_\mu(x) = X\,U_\mu(x)$ the change of $-2\,\mathrm{Re}\,\zeta^\dagger\,\delta D\,\eta$ is $\mathrm{Re\ tr}(X M_\mu(x))$ with

$$M_\mu(x) = -2c\sum_s\sum_{\text{spin}}\big[U_\mu(x)(1-\gamma_\mu)\eta_s(x+\hat\mu)\big]\zeta_s(x)^\dagger + 2c\sum_s\sum_{\text{spin}}\big[\gamma_5\eta_s(x)\big]\big[U_\mu(x)(1-\gamma_\mu)\gamma_5\zeta_s(x+\hat\mu)\big]^\dagger. \tag{4.1}$$

The kernel gets $U(1-\gamma_\mu)\eta(x+\hat\mu)$ from the forward leg of the hop kernel with $\kappa=-1$ (so the leg has coefficient $+1$), and the second bracket the same way from $\gamma_5\zeta$. Then the outer products, colour by colour:

$$(ab^\dagger)_{ij} = a_i\overline{b_j}, \qquad \mathrm{Re} = a_i^r b_j^r + a_i^i b_j^i, \quad \mathrm{Im} = a_i^i b_j^r - a_i^r b_j^i. \tag{4.2}$$

A thread per site, a loop over μ and s; M in the layout of the gauge fields (Sec. 5).

5 The gauge field

5.1 Layout

Links and fields of matrices (staples, Σ, forces) are whole 3×3 complex matrices, 18 reals per direction, 72 per site, in blocks of 32 sites (Eq. (1.1) with $N_c = 72$). All in double precision.

5.2 `stout_step_kernel`: $V = e^{iQ}U$

The staple sum and Q:

$$C_\mu(x) = \sum_{\nu\neq\mu}\rho_{\mu\nu}\Big[U_\nu(x)U_\mu(x+\hat\nu)U_\nu(x+\hat\mu)^\dagger + U_\nu(x-\hat\nu)^\dagger U_\mu(x-\hat\nu)U_\nu(x-\hat\nu+\hat\mu)\Big], \tag{5.1}$$

$$\Omega = C\,U^\dagger, \tag{5.2}$$

$$iQ = \tfrac{1}{2}(\Omega-\Omega^\dagger) - \tfrac{1}{6}\,\mathrm{tr}(\Omega-\Omega^\dagger)\cdot 1 \qquad \text{(traceless anti-Hermitian)}. \tag{5.3}$$

The exponential by Cayley–Hamilton. Q is Hermitian and traceless, so

$$Q^3 = c_1 Q + c_0\cdot 1, \qquad c_0 = \det Q = \tfrac{1}{3}\,\mathrm{tr}\,Q^3, \qquad c_1 = \tfrac{1}{2}\,\mathrm{tr}\,Q^2, \tag{5.4}$$

and every power of Q reduces to $1, Q, Q^2$:

$$e^{iQ} = f_0\cdot 1 + f_1 Q + f_2 Q^2. \tag{5.5}$$

With the eigenvalues parametrised as

$$\theta = \arccos\frac{c_0}{c_0^{\max}}, \qquad c_0^{\max} = 2\Big(\frac{c_1}{3}\Big)^{3/2}, \tag{5.6}$$

$$u = \sqrt{c_1/3}\,\cos\frac{\theta}{3}, \qquad w = \sqrt{c_1}\,\sin\frac{\theta}{3}, \tag{5.7}$$

Morningstar and Peardon find $f_j = h_j/(9u^2-w^2)$ with

$$h_0 = (u^2-w^2)e^{2iu} + e^{-iu}\big[8u^2\cos w + 2iu(3u^2+w^2)\,\xi_0(w)\big], \tag{5.8}$$

$$h_1 = 2u\,e^{2iu} - e^{-iu}\big[2u\cos w - i(3u^2-w^2)\,\xi_0(w)\big], \tag{5.9}$$

$$h_2 = e^{2iu} - e^{-iu}\big[\cos w + 3iu\,\xi_0(w)\big], \qquad \xi_0(w) = \frac{\sin w}{w}. \tag{5.10}$$

The kernel stores iQ and $(iQ)^2$ and uses $Q = -i(iQ)$, $Q^2 = -(iQ)^2$. When $|iQ|^2 \le 10^{-10}$ it takes $e^{iQ} = 1$. A thread for each (site, direction).

5.3 `stout_chain_local_kernel` and `stout_chain_staples_kernel`

The force on the thin links from Σ', the force on the smeared ones (Morningstar–Peardon eqs. (70)–(75)):

$$\Lambda = \tfrac{1}{2}(\Gamma + \Gamma^\dagger) - \tfrac{1}{6}\,\mathrm{tr}(\Gamma + \Gamma^\dagger), \tag{5.11}$$

$$\Gamma = \mathrm{tr}(\Sigma' B_1 U)\, Q + \mathrm{tr}(\Sigma' B_2 U)\, Q^2 + f_1\, U\Sigma' + f_2\, Q\, U\Sigma' + f_2\, U\Sigma' Q, \tag{5.12}$$

$$B_i = b_{i0} + b_{i1} Q + b_{i2} Q^2, \tag{5.13}$$

with b_{ij} derivatives of the f_j by c_0, c_1 (in the code, `r01`..`r22` and the `b`'s; $\xi_1(w) = (w\cos w - \sin w)/w^3$ by its series for $w < \frac{1}{4}$). Then

$$\Sigma = \Sigma'\, e^{iQ} + C^\dagger\, i\Lambda + \sum(\text{terms of the staples that contain } U_\mu(x)), \tag{5.14}$$

$$i\Theta = i\Lambda\, U. \tag{5.15}$$

The first kernel makes the first two terms and $i\Theta$; the second adds the six staple terms of each plane, which read $i\Theta$ at the neighbours. On several GPUs $i\Theta$ is extended between the two.

5.4 `gauge_force_kernel` and `gauge_traces_kernel`

$$F_\mu(x) = -\frac{\beta}{3}\,\mathrm{TA}\Big[U_\mu(x)\,\Sigma_\mu(x)^\dagger\Big], \tag{5.16}$$

$$\Sigma_\mu = c_{\mathrm{plaq}} \sum(\text{2 staples per plane}) + c_{\mathrm{rect}} \sum(\text{6 rectangle staples per plane}), \tag{5.17}$$

each staple a product of links along a path (`walk`). The traces kernel sums per site Re tr of the 6 plaquettes and 12 rectangles; the action follows on the host after the sum over sites.

5.5 `links_exp_update_kernel`: $U \leftarrow e^{\epsilon H} U$

The Taylor series to order n in nested (Horner) form:

$$e^{\epsilon H} U \approx \sum_{k=0}^{n} \frac{(\epsilon H)^k}{k!} U \tag{5.18}$$

$$= U + \epsilon H\Big(U + \frac{\epsilon H}{2}\Big(U + \frac{\epsilon H}{3}(\cdots)\Big)\Big), \tag{5.19}$$

computed from the inside out:

$$u^{(0)} = U, \qquad u^{(k+1)} = U + \frac{\epsilon}{n-k}\, H\, u^{(k)}, \qquad k = 0, \ldots, n-1. \tag{5.20}$$

Then reunitarised: $r_0 \to r_0/|r_0|$, $r_1 \to (r_1 - (\bar{r}_0 \cdot r_1) r_0)/|\cdot|$, $r_2 = \overline{r_0 \times r_1}$.

5.6 Other small kernels

`links_axpy` ($y \mathrel{+}= ax$), `links_ta` ($s\,\mathrm{TA}(m)$), `links_mul` (ab or $a^\dagger b$), `links_norm2` (per site $\sum$ reals^2, for $\frac{1}{2}\sum \mathrm{tr}\, P^2$).

6 The multigrid: P

6.1 What P holds

An aggregate keeps the whole fifth dimension and is split by the chirality of $\Gamma_5 = \gamma_5 R$, $(R\psi)_s = \psi_{L_s-1-s}$. For a test vector v:

$$v^{(c)} = \tfrac{1}{2}(1 + (-1)^c \Gamma_5) v, \qquad c = 0, 1. \tag{6.1}$$

At fifth coordinate s:

$$v_s^{(c)} = \tfrac{1}{2}\big(v_s + (-1)^c \gamma_5 v_{L_s-1-s}\big). \tag{6.2}$$

Now look at the mirror $L_s - 1 - s$:

$$\begin{aligned} v_{L_s-1-s}^{(c)} &= \tfrac{1}{2}\big(v_{L_s-1-s} + (-1)^c \gamma_5 v_s\big) && (6.3)\\ &= (-1)^c \gamma_5 \cdot \tfrac{1}{2}\big((-1)^c \gamma_5 v_{L_s-1-s} + v_s\big) \qquad (\gamma_5^2 = 1,\ ((-1)^c)^2 = 1) && (6.4)\\ &= (-1)^c \gamma_5\, v_s^{(c)}. && (6.5) \end{aligned}$$

So the half $s < L_s/2$ determines the rest, and P is stored folded: $12L_s$ reals per (site, test vector, chirality) instead of $24L_s$, in single or half precision. `fold_fill_kernel` writes $\frac{1}{2}(v_s \pm \gamma_5 v_{L_s-1-s})$ for $s < L_s/2$.

6.2 `fold_restrict_kernel`: coarse $= P^\dagger$fine

For coarse site X, test vector i, chirality c:

$$\begin{aligned} \mathrm{coarse}(X, cn+i) &= \sum_{x\in X} \sum_{s=0}^{L_s-1} P(x,i,c)_s^\dagger\, f_s(x) && (6.6)\\ &= \sum_{x\in X} \sum_{s<L_s/2} \Big[P_s^\dagger f_s + P_{L_s-1-s}^\dagger f_{L_s-1-s}\Big] && (6.7)\\ &= \sum_{x\in X} \sum_{s<L_s/2} \Big[P_s^\dagger f_s + \big((-1)^c \gamma_5 P_s\big)^\dagger f_{L_s-1-s}\Big] && (6.8)\\ &= \sum_{x\in X} \sum_{s<L_s/2} P_s^\dagger \Big[f_s + (-1)^c \gamma_5 f_{L_s-1-s}\Big] \qquad (\gamma_5^\dagger = \gamma_5). && (6.9) \end{aligned}$$

The last line is what the kernel computes: one read of the folded P per pair (s, L_s-1-s). A thread for each (coarse site, component), up to 8 vectors per read of P, which is most of the cost.

6.3 `fold_prolong_kernel`: fine $= P$coarse

$$\begin{aligned} f_s(x) &= \sum_{c,i} P(x,i,c)_s\, \mathrm{coarse}(X, cn+i), && (6.10)\\ f_{L_s-1-s}(x) &= \sum_{c,i} (-1)^c \gamma_5 P(x,i,c)_s\, \mathrm{coarse}(X, cn+i), && (6.11) \end{aligned}$$

a thread for each (fine site, $s < L_s/2$, spin, colour), writing s and its mirror.

6.4 Orthonormalising P

Per (aggregate, chirality), the n vectors are made orthonormal over the aggregate. With $W = [w_1 \dots w_n]$:

$$G = W^\dagger W = LL^\dagger \text{ (Cholesky)}, \qquad Q = W\,(L^\dagger)^{-1} \ \Rightarrow\ Q^\dagger Q = L^{-1} G L^{-\dagger} = 1. \tag{6.12}$$

`fold_orthonormalise_cholesky_kernel` does this in double precision. Cholesky squares the condition number, so where a pivot is below 10^{-12} of its diagonal the kernel gives up on that site and `fold_orthonormalise_kernel` redoes it by modified Gram–Schmidt. The folded P holds half of every inner product, so the result is scaled by $1/\sqrt{2}$.

7 The multigrid: the coarse operator

7.1 Probing: `hop_split_kernel`, `coarse_units`, `coarse_columns`

With e_j the unit vector of component j on every coarse site:

$$\phi_j = P\, e_j, \tag{7.1}$$

$$A(X)_{\cdot j} = [P^\dagger\, C\phi_j](X) + \sum_{\text{legs inside the block}} [P^\dagger \log \phi_j](X), \tag{7.2}$$

$$B_d(X)_{\cdot j} = [P^\dagger \log_d \phi_j](X) \quad \text{on the sites whose leg } d \text{ leaves the block.} \tag{7.3}$$

`hop_split_kernel` computes one leg and sorts it: into `cross` where it comes from another block, added to `inside` elsewhere. The restriction of `cross` is column j of B_d; `coarse_columns` writes it straight into the matrices, in the precision they are kept in. Several j at a time, as the vectors of the fields.

7.2 `coarse_pass_kernel`

One kernel for every part of the coarse operator:

$$\mathrm{out}(X) = M(X)\Big[\mathrm{base}(X) + A(X)\, d(X) \pm \sum_d B_d(X)\, h\big(N[d][X]\big)\Big], \tag{7.4}$$

each term optional. Choices give D_c, the Schur complement on the even sites $S = A_{ee} - B_{eo}A_{oo}^{-1}B_{oe}$, its source and solution, the block inverses. $D_c^\dagger = \Gamma_5 D_c \Gamma_5$ (Γ_5: -1 on the second half of the components) is the same kernel with a flag.

Threads. A block per (coarse site, vector); a warp per row; the lanes take pairs of columns $(2\ell, 2\ell+1), (2\ell+64, \dots)$, so a warp reads a row of a matrix as one stretch in 16-byte words. The fields at X and its 8 neighbours are read once into shared memory. The sum of a row is the lanes' parts added by a fixed butterfly, so the result does not depend on timing.

7.3 `blocks_inverse_kernel`: Gauss–Jordan

For each site, $[A \,|\, 1] \to [1 \,|\, A^{-1}]$, thread r holding row r:

$$\text{for } k = 0, \dots, \dim - 1: \tag{7.5}$$

$$p = \arg\max_{r \ge k} |a_{rk}|, \ \text{swap rows } k, p, \tag{7.6}$$

$$\mathrm{row}_k \leftarrow \mathrm{row}_k / a_{kk}, \tag{7.7}$$

$$\mathrm{row}_r \leftarrow \mathrm{row}_r - a_{rk}\, \mathrm{row}_k \quad (r \ne k). \tag{7.8}$$

A zero pivot sets a failure flag (a singular block).

7.4 `blocks_abs_inverse_gamma5_kernel`: $|H|^{-1}$ by Jacobi

H = Hermitian part of $A\Gamma_5$. The cyclic Jacobi method: for each pair (p, q),

$$\text{phase: } h_{pq} = |h_{pq}| e^{i\varphi} \ \Rightarrow\ \text{multiply column } q \text{ by } e^{-i\varphi},\ \text{row } q \text{ by } e^{i\varphi}\ (h_{pq} \text{ now real}), \tag{7.9}$$

$$\text{rotation: } \tan 2\vartheta = \frac{2h_{pq}}{h_{qq} - h_{pp}},\ \text{rotate rows and columns } p, q \ \Rightarrow\ h_{pq} = 0, \tag{7.10}$$

repeated until the off-diagonal part vanishes, accumulating V. Then

$$|H|^{-1} = V\, |\Lambda|^{-1} V^\dagger, \tag{7.11}$$

with eigenvalues below a cutoff times the largest left out (they come from dependent vectors).

7.5 The exact adjoint: `coarse_adjoint_kernel`

$$(A^\dagger u)(Y) = A(Y)^\dagger u(Y) + \sum_d B_d(N[\bar{d}][Y])^\dagger \, u(N[\bar{d}][Y]) \tag{7.12}$$

(derived in the other note); the kernel writes $B_d(X)^\dagger$ at the place of Y, entry by entry: $(m^\dagger)_{ij} = \overline{m_{ji}}$.

8 The levels below the coarse lattice

`level_basis` :	$Q_{\text{block}} =$ Gram–Schmidt twice of the parts of the test vectors on a super-block and a half,	(8.1)
`level_restrict` :	$y = Q^\dagger v$ (a warp per output, lanes along the sum),	(8.2)
`level_prolong` :	$v = Qy$ or $v \mathrel{+}= Qy$,	(8.3)
`level_sources` :	$e_j = 1$ at component j of the sites of one colour,	(8.4)
`level_probe` :	$A(t)_{\cdot j} = r_j(t)$ if t has the colour, $B_d(t)_{\cdot j} = r_j(t)$ if $N[d][t]$ has it,	(8.5)
`level_dense` :	the last level as one $n \times n$ matrix, $n =$ sites $\times$ components,	(8.6)
`dense_apply` :	out $= M\, r$.	(8.7)

Why probing works: the sources of one colour are at least 3 apart in every direction, so

$$r_j(t) = \sum_{t'} D(t,t') e_j(t') = D(t,t'_*)\,\hat{e}_j \quad \text{with } t'_* \in \{t,\ t \pm \hat{\mu}\} \text{ the only source in reach.} \tag{8.8}$$

The response at t comes from one source only, and which one tells whether it is $A(t)$ or a $B_d(t)$. The dense inverse (LU) and $|H|^{-1}$ of the last level are done by cuSOLVER (`dense.cu`).

9 Linear algebra: `blas.cuh`

9.1 Fused operations

A solver step usually needs several updates and a norm at once, e.g.

$$r \leftarrow r - \alpha A p, \qquad |r|^2. \tag{9.1}$$

`blas_map_kernel` and `blas_reduce_kernel` take the operation as a functor (`Op`) and do it in one pass over the fields, every vector of a set at once.

9.2 Deterministic reductions

A sum over N numbers is done in two stages, both in a fixed order:

$$s_b = \sum_{p \in \text{block } b} x_p \quad \text{(a tree inside each CUDA block, in double precision)}, \tag{9.2}$$

$$S = \sum_b s_b \quad (\texttt{blas_finish_kernel},\ \text{one block, a fixed tree}). \tag{9.3}$$

No atomic additions, so the result does not depend on which block finishes first: runs repeat bit for bit. Only the final numbers go to the host.

9.3 Sets of vectors

`blas_gram_kernel`: all $\langle x_i, y_j \rangle$ in one pass. `blas_rotate_kernel`: $x \leftarrow xC$ in place for a small C. `blas_combine_kernel`: $\text{out}_j = \sum_i x_i c_{ij}$ for larger sets, the outputs of a chunk in registers.

Appendix: kernels removed

Extracted from Bridge++ with the rest but never called by the library, and removed on 2026-10-24 with the C tests that exercised only them: the Schwarz domain decomposition (`dd.cuh`), an older restriction, prolongation and coarse operator (`transfer.cuh`, `mdwf_transfer.cuh`, `coarse.cuh`, `coarse_eo.cuh`), multi-right-hand-side Wilson (`mrhs.cuh`), APE smearing and a plaquette (`gauge.cuh`), the Dirac/chiral conversion, a plain real `axpy`, and an unfinished Wilson face packing.